Rendering and loading

8A. Rendering & Loading

Questions: 5 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft

---

Q 8.1 What is the Critical Rendering Path and how do you optimize it?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Microsoft

The Critical Rendering Path (CRP) is the sequence of steps the browser takes to convert HTML, CSS, and JavaScript into pixels on screen. Understanding it is the foundation for all performance optimization — every technique in this chapter targets a specific step in this path.

The path: HTML → DOM → CSS → CSSOM → Render Tree → Layout → Paint → Composite (covered in detail in Chapter 5, Q5.1 and Q5.7).

What makes it "critical":

The word "critical" refers to the resources that block rendering. The browser can't paint anything until it has built both the DOM and the CSSOM. CSS is render-blocking by default. JavaScript (without defer/async) is parser-blocking — it stops DOM construction entirely.

Optimizing the CRP means reducing three things:

1. Critical resource count — how many resources block rendering

HTML Copy

```html
<!-- Before: 3 render-blocking resources -->
<link rel="stylesheet" href="/main.css" />
<link rel="stylesheet" href="/vendor.css" />
<script src="/analytics.js"></script>

<!-- After: 1 render-blocking resource, rest deferred -->
<style>/* Inlined critical CSS - only above-the-fold styles */</style>
<link rel="preload" href="/main.css" as="style" onload="this.rel='styleshe
<script defer src="/analytics.js"></script>
```

2. Critical path length — the number of sequential round trips

If CSS file A imports CSS file B, the browser must download A, parse it, discover the import, then download B. That's two sequential round trips. Inlining critical CSS or using `preload` eliminates the chain.

3. Critical bytes — total size of render-blocking resources

Minify CSS, remove unused styles (PurgeCSS), split CSS per route, and inline only the above-the-fold critical CSS (~14KB is the ideal target — fits in the first TCP congestion window).

Practical optimization sequence:

1. Inline critical CSS in `<head>` (above-the-fold styles only)
2. Defer all non-critical CSS with `preload` + `onload` swap
3. Add `defer` or `async` to all scripts
4. Preconnect to critical origins (`<link rel="preconnect">`)
5. Preload critical resources (fonts, hero images)
6. Minimize DOM depth and complexity

Follow-up: "How do you identify what's on the critical path?"

also explicitly calls out render-blocking resources in its audit.

Q 8.2 What are Core Web Vitals? How do you measure and improve LCP, INP, and CLS?

Difficulty: Mid **Asked at:** Every company — this is the most frequently asked performance question

Core Web Vitals are Google's metrics for user experience. They measure three things: how fast content appears (LCP), how responsive the page is to interaction (INP), and how visually stable the layout is (CLS). Google uses them as a ranking signal for search results.

LCP — Largest Contentful Paint

Measures when the largest visible content element finishes rendering. This is usually the hero image, the main heading, or a large text block.

Good: < 2.5 seconds. Poor: > 4 seconds.

Common causes of bad LCP:

- Slow server response (high TTFB) — fix with CDN, caching, SSR
- Render-blocking CSS/JS — fix with critical CSS inlining, defer scripts
- Slow resource loading (hero image) — fix with `<link rel="preload">`, responsive images, modern formats (WebP/AVIF)
- Client-side rendering — the largest element can't render until JS executes and API calls complete

HTML Copy

```
<!-- Preload the LCP image so it starts downloading immediately -->
<link rel="preload" href="/hero.webp" as="image" fetchpriority="high" />
```

```
  <source type="image/webp" srcset="/hero.webp" />
  <img src="/hero.jpg" alt="Hero" width="1200" height="600" fetchpriority=
</picture>
```

INP — Interaction to Next Paint

Measures the delay between user interaction (click, tap, keypress) and the next visual update. Replaced FID (First Input Delay) in March 2024. Unlike FID which only measured the first interaction, INP measures all interactions throughout the page lifecycle and reports the worst one.

Good: < 200ms. Poor: > 500ms.

Common causes of bad INP:

- Long JavaScript tasks blocking the main thread — fix with code splitting, Web Workers, `scheduler.yield()`
- Expensive re-renders in React — fix with memoization, virtualization, `useTransition`
- Synchronous layout thrashing — fix by batching DOM reads/writes

JAVASCRIPT Copy

```javascript
// Break long tasks to keep the main thread responsive
async function processLargeList(items) {
  for (let i = 0; i < items.length; i++) {
    processItem(items[i]);

    // Yield to the main thread every 50 items
    if (i % 50 === 0) {
      await scheduler.yield();  // Browser can handle user input here
    }
  }
}
```

CLS — Cumulative Layout Shift

everything down" problem.

Good: < 0.1. Poor: > 0.25.

Common causes of bad CLS:

- Images without `width` and `height` attributes — browser doesn't know the dimensions until the image loads
- Fonts loading and changing text size (FOUT) — fix with `font-display: optional` or preloading fonts
- Dynamic content injected above existing content — ads, banners, cookie notices
- Lazy-loaded content without placeholder space

HTML Copy

```
<!— Always set width/height on images - browser reserves space —>
<img src="/product.jpg" width="400" height="300" alt="Product" loading="la

<!— Reserve space for dynamic content —>
<div style="min-height: 250px;">
  <!— Ad will load here without pushing content down —>
</div>
```

Measuring Core Web Vitals:

- **Lab data:** Lighthouse (Chrome DevTools), PageSpeed Insights, WebPageTest
- **Field data (real users):** Chrome User Experience Report (CrUX), `web-vitals` library, RUM tools (Sentry, Datadog)

JAVASCRIPT Copy

```
// Measuring in production with the web-vitals library
import { onLCP, onINP, onCLS } from 'web-vitals';

onLCP(metric => sendToAnalytics('LCP', metric.value));
```

Lab data tells you what's possible. Field data tells you what's actually happening for your users. Always prioritize field data — a page can score 100 in Lighthouse and still have terrible INP in the field because real users interact differently than synthetic tests.

Follow-up: "Which vital would you prioritize first?"

LCP. It's the most impactful for user perception and the easiest to improve. Preloading the hero image and inlining critical CSS often gets you 80% of the way. INP requires deeper architectural work (code splitting, optimizing React renders). CLS is usually fixed by adding image dimensions and reserving space for dynamic content.

Q 8.3 How does bundle optimization work — tree shaking, code splitting, and lazy loading?

Difficulty: Mid **Asked at:** Every company

These three techniques reduce the amount of JavaScript the browser needs to download and execute. They solve different problems.

Tree shaking — removing dead code

Tree shaking analyzes your import graph and removes exported code that's never imported anywhere. It works because ES Modules (`import` / `export`) are statically analyzable — the bundler knows at build time exactly what's used.

JAVASCRIPT Copy

```
// math.js - exports 4 functions
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }
export function multiply(a, b) { return a * b; }
export function divide(a, b) { return a / b; }

// app.js - only uses add
import { add } from './math';
```

Tree shaking fails when:

- You use `import * as math from './math'` — bundler can't tell what's used
- The library has side effects (code that runs on import, not just exports)
- You use CommonJS (`require`) instead of ESM (`import`)

Check your bundle for dead code: `npx vite-bundle-visualizer` or `webpack-bundle-analyzer`.

Code splitting — loading only what's needed for the current route

Instead of one giant bundle with every route's code, split into per-route chunks that load on demand.

JSX Copy

```jsx
import { lazy, Suspense } from 'react';

// These components are in separate chunks — only downloaded when the rout
const Dashboard = lazy(() => import('./pages/Dashboard'));
const Settings = lazy(() => import('./pages/Settings'));
const Analytics = lazy(() => import('./pages/Analytics'));

function App() {
  return (
    <Suspense fallback={<PageLoader />}>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/settings" element={<Settings />} />
        <Route path="/analytics" element={<Analytics />} />
      </Routes>
    </Suspense>
  );
}
```

Lazy loading — deferring non-critical resources

Lazy loading applies to any resource, not just code. Images, components, and even data can be lazy-loaded.

HTML Copy

```
<!-- Native lazy loading for images — no JavaScript needed -->
<img src="/product.jpg" loading="lazy" alt="Product" width="400" height="3

<!-- Never lazy-load above-the-fold images — they should load immediately
<img src="/hero.jpg" alt="Hero" width="1200" height="600" fetchpriority="h
```

JAVASCRIPT Copy

```
// Lazy load a heavy library only when needed
async function handleExport() {
  const { exportToExcel } = await import('./exportUtils');
  exportToExcel(data);
}
// The export library only downloads when the user clicks "Export"
```

The relationship between the three:

Tree shaking makes each chunk smaller (removes unused code within a chunk). Code splitting creates separate chunks (so you only load the route you need). Lazy loading defers when chunks are downloaded (on route navigation, on user action, on visibility).

Follow-up: "How do you analyze what's in your bundle?"

BASH Copy

```
# Vite
npx vite-bundle-visualizer

# Webpack
npx webpack-bundle-analyzer stats.json
```

```
ANALYZE=true next build  # with @next/bundle-analyzer
```

These tools generate a treemap visualization showing every module in your bundle and its size. Look for: large dependencies you didn't know about, duplicate packages (two versions of the same library), and libraries you imported but barely use (importing all of `lodash` for one function).

Q 8.4 How do you optimize images? Explain formats, responsive images, and lazy loading.

Difficulty: Mid **Asked at:** Flipkart, Amazon, Swiggy, Google

Images typically account for 50-70% of a page's total weight. Optimizing images is often the single highest-impact performance improvement you can make.

Modern image formats:

- **JPEG:** Good for photographs. No transparency. Widely supported.
- **PNG:** Lossless, supports transparency. Large files. Use only when you need transparency with hard edges (logos, icons).
- **WebP:** 25-35% smaller than JPEG at equivalent quality. Supports transparency. Full browser support since 2023.
- **AVIF:** 40-50% smaller than JPEG. Best compression of any format. Slower to encode. Browser support at ~95% in 2026.

Always serve AVIF with WebP fallback and JPEG as the last resort:

HTML Copy

```html
<picture>
  <source type="image/avif" srcset="/product.avif" />
  <source type="image/webp" srcset="/product.webp" />
```

Responsive images — serve the right size for the device:

A 4K desktop doesn't need the same image as a mobile phone. Serving a 2400px image to a 375px screen wastes bandwidth.

HTML Copy

```
<img
  src="/product-800.jpg"
  srcset="
    /product-400.jpg 400w,
    /product-800.jpg 800w,
    /product-1200.jpg 1200w,
    /product-1600.jpg 1600w
  "
  sizes="(max-width: 600px) 100vw, (max-width: 1200px) 50vw, 800px"
  alt="Product"
  loading="lazy"
  width="800"
  height="600"
/>
```

`srcset` tells the browser which sizes are available. `sizes` tells it how much viewport space the image will occupy. The browser combines this with the device pixel ratio to pick the optimal image.

Blur placeholder pattern:

Show a tiny (20-30 byte) blurred version of the image while the full version loads. This eliminates the jarring pop-in of images appearing from nothing.

HTML Copy

```
<!-- Inline a tiny 10x10 blurred version as base64 -->
<div style="background: url( ... ) no-repeat
```

Next.js `<Image>` component does this automatically with the `placeholder="blur"` prop.

The optimization checklist:

1. Use AVIF/WebP formats (40-50% size reduction)
2. Serve responsive sizes (`srcset` + `sizes`)
3. Lazy load below-the-fold images (`loading="lazy"`)
4. Set `fetchpriority="high"` on the hero/LCP image
5. Always include `width` and `height` (prevents CLS)
6. Use blur placeholders for perceived performance
7. Compress at the right quality (80-85% for JPEG is the sweet spot)

Follow-up: "Should you self-host images or use a CDN/image service?"

Image CDNs (Cloudinary, Imgix, Cloudflare Images) are almost always the right choice for production. They automatically serve the optimal format (AVIF if supported, WebP fallback), resize on-the-fly based on URL parameters, cache at the edge, and handle responsive images without you generating multiple versions at build time. The URL-based API is simpler than managing 5 variants of every image.

Q 8.5 How do font loading strategies work? Explain FOIT, FOUT, and `font-display`.

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart

Fonts are a hidden performance problem. The browser discovers fonts late (during CSS parsing, not HTML parsing), they're often large (100-300KB for a family), and they block text rendering by default.

(or after a timeout), text appears. If the font fails, fallback text shows.

FOUT (Flash of Unstyled Text): Text renders immediately in a fallback system font. When the custom font loads, text re-renders with the new font, causing a visible shift. Less jarring than invisible text, but still a layout shift.

`font-display` **controls which behavior you get:**

CSS Copy

```css
@font-face {
  font-family: 'Inter';
  src: url('/fonts/Inter.woff2') format('woff2');
  font-display: swap;  /* Show fallback immediately, swap when loaded */
}
```

JAVASCRIPT Copy

```
Value     | Behavior
  ————————┼————————————————————————————————————————————
  auto    | Browser default (usually FOIT)
  block   | FOIT — invisible text for up to 3 seconds, then fallback
  swap    | FOUT — fallback immediately, swap when font loads
  fallback| Very short FOIT (~100ms), then fallback. If font loads
          | within ~3s, swap. If not, keep fallback forever.
  optional| Very short FOIT (~100ms), then fallback. Browser may
          | decide not to use the custom font at all if it's slow.
          | Best for performance — zero CLS if font doesn't load.
```

The recommended approach in 2026:

HTML Copy

```html
<!— 1. Preload the font so it downloads early —>
<link rel="preload" href="/fonts/Inter-Regular.woff2" as="font"
      type="font/woff2" crossorigin />

<!— 2. Use font-display: swap or optional —>
```

```
    font-family: 'Inter';
    src: url('/fonts/Inter-Regular.woff2') format('woff2');
    font-display: swap;
    font-weight: 400;
  }

  @font-face {
    font-family: 'Inter';
    src: url('/fonts/Inter-Bold.woff2') format('woff2');
    font-display: swap;
    font-weight: 700;
  }
</style>
```

Self-hosting vs Google Fonts:

Self-hosting is recommended in 2026. Google Fonts requires a connection to `fonts.googleapis.com` (CSS) and `fonts.gstatic.com` (font files) — that's two extra origins, each needing DNS + TCP + TLS. Self-hosting from your own CDN eliminates these connections. Download the font files, convert to woff2, and host them alongside your CSS.

Variable fonts — one file instead of many:

A variable font contains multiple weights and styles in a single file. Instead of loading `Inter-Regular.woff2` (90KB) + `Inter-Bold.woff2` (90KB) + `Inter-Italic.woff2` (95KB), you load one `Inter-Variable.woff2` (~200KB) that contains all variants. Fewer requests, and the total size is often smaller than the sum of individual files.

css Copy

```css
@font-face {
  font-family: 'Inter';
  src: url('/fonts/Inter-Variable.woff2') format('woff2-variations');
  font-weight: 100 900;  /* Supports any weight in this range */
  font-display: swap;
}

h1 { font-weight: 700; }
```

Follow-up: "How do you reduce the CLS caused by font swapping?"

Match the fallback font's metrics (line height, letter spacing, font size) to the custom font as closely as possible. The `size-adjust`, `ascent-override`, `descent-override`, and `line-gap-override` CSS descriptors let you tune the fallback font to match. Next.js `next/font` does this automatically.

QUICK CHECK — no marks, just for you.

1. INP (Interaction to Next Paint) replaced FID as a Core Web Vital. What is the key measurement difference between them?

- ○ FID measured only the first interaction, while INP measures all interactions across the page lifecycle and reports the worst one
- ○ FID measured paint timing, while INP measures how long the server takes to respond to the first request
- ○ FID measured interactions on mobile only, while INP measures interactions on both mobile and desktop
- ○ FID measured all interactions and averaged them, while INP reports only the first interaction after load

Check answer

2. Tree shaking, code splitting, and lazy loading are often confused. According to the lesson, which statement correctly describes what each one does?

- ○ All three are different names for the same bundler optimization, differing only by which tool implements them
- ○ Tree shaking defers download until user action, code splitting removes dead code, and lazy loading merges chunks into a single bundle

Tree shaking creates separate chunks per route, code splitting removes unused exports, and lazy loading compresses the chunks before transfer

Check answer

3. One of the three things you reduce when optimizing the Critical Rendering Path is "critical path length". What does that specifically measure?

- The count of individual resources that block rendering before First Contentful Paint
- The depth of the DOM tree that the browser must traverse when building the render tree
- The total byte size of all render-blocking CSS and JavaScript that must be downloaded
- The number of sequential round trips needed to fetch render-blocking resources, such as a CSS file that imports another CSS file

Check answer

Previous
Modern rendering patterns

Next
Runtime and network

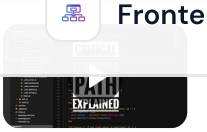
LEARN MORE

ARTICLES & DOCS

Critical rendering path (MDN)
MDN

VIDEOS

Frontend System Desi...



Critical rendering path explained

YouTube



Critical rendering path walkthrough

YouTube



CRP — frontend interview question

YouTube

Hand-picked references for this lesson.